

# claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\workflows\wf_d251ad6c-472\agent-a4e34dd1829e7db80.jsonl`  
- SHA-256: `f51cc90db4350c992e86ccc8544d81a991eb1abc808e73d77407381b8a615a82`  
- Source modified: `2026-06-08T08:45:41+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_d251ad6c-472`  
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`
```

## Transcript

### 1. user (2026-06-08T08:43:41.060Z)

LENS = PRAGMATIST. Propose the SMALLEST set of changes to make the collector ingest the rally-annotator's PARTIAL human CSV TODAY through the existing import-local path, validate it, and export labels the indexer eval can consume. Favor shipping this week over ideal architecture. Be concrete about which columns map where and how to handle partial coverage + fractional rally numbers + missing end\_time.

Ground every recommendation in these subsystem FINDINGS (cite files). Do not invent files or fields that the findings do not support.

FINDINGS:

```
[  
  {  
    "subsystem": "Sports Data Collector - Local/Manual Ingestion CLI",  
    "summary": "The sports-data-collector provides a manual ingestion pipeline (`import-local` command) that accepts local video files paired with JSON or CSV annotation files in a sport-specific rally/point/delivery format. It parses annotations, validates temporal consistency (no overlaps, ordered progression), registers metadata + segments in SQLite, and exports curated data as indexer-ready eval CSVs containing only (start, end) decimal-second intervals per video.",  
    "key_facts": [  
      "CLI entry: src/cli/curate.py:844-961 (main function with subparser setup); import-local command registered line 854-863",  
      "import_local() method: src/cli/curate.py:52-155; accepts --sport (required, lowercase), --video-path (required), --annotations-path (required), --match-name (optional, defaults to video basename), --video-id (optional, defaults to lowercased match_name), --license (default='Proprietary'), --fps (default=30.0), --resolution (default='1080p')",  
      "_parse_local_annotations() method: src/cli/curate.py:156-302; accepts .json or .csv files; normalizes CSV headers to lowercase; raises ValueError if unsupported extension",  
      "Badminton schema: src/core/schemas/badminton.py:4-22; required fields: video_id, rally_number (int), start_time (float), end_time (float); optional: score_before, score_after, shots_count (int), ending_reason, last_shot_outcome; validates end_time > start_time",  
      "CSV parsing: src/cli/curate.py:232-298; uses csv.DictReader with normalized lowercase headers; safe_int() with idx+1 default for rally_number; safe_float_required() for start_time, end_time (raises ValueError if missing/empty); gracefully degrades optional fields to None",  
      "JSON parsing: src/cli/curate.py:161-231; expects list of objects; uses identical safe_int/safe_float logic; treats missing required fields (start_time, end_time) as ValueError",  
      "Validation pipeline: src/cli/curate.py:88-106 calls validator.validate_badminton_rallies() at src/core/validator.py:29-57; checks: no zero/negative duration (start_time < end_time), no temporal overlaps between sequential rallies (sorted by rally_number), chronological progression; returns (bool, List[str]) errors",  
      "DB insertion: src/cli/curate.py:123-136 inserts VideoMetadata (status=CURATED for manual imports, bypassing staging) + rallies; db.insert_badminton_rallies() at src/core/db.py:210-237 performs UPSERT (DELETE then INSERT per video_id)",  
      "Export format: export_eval_set() at src/cli/curate.py:440-536; emits per-video CSV with
```

```

header [start,end] + decimal-second tuples (queried via get_segment_intervals() at
src/core/db.py:527-546); manifest.json pairs CSVs with video metadata (local_path, video_url,
fps, resolution, etc.)",
    "License handling: LicenseType enum at src/core/schemas/base.py:12-22 (MIT, Apache-2.0,
BSD, CC0, CC-BY, CC-BY-NC, CC-BY-NC-SA, Public-Domain, Proprietary, Unknown);
validator.check_license_commercial() at src/core/validator.py:24-27 checks against whitelist
(config default: MIT, Apache-2.0, BSD, CC0, CC-BY, Public-Domain, Proprietary)",
    "CurationStatus enum: src/core/schemas/base.py:24-27 (STAGING, CURATED, REJECTED); manual
imports set status=CURATED directly (line 117)",
    "Rally-annotator CSV format compatibility: docs/CSV_FORMAT.md specifies columns
(rally_number, start_time, end_time, ending_reason, sport); decimal seconds, 1-based rally
numbers; sports-data-collector import-local reads this directly (compatible via DictReader
header matching)"
],
    "contracts": [
        "import-local CLI flags: --sport (required:
badminton|tennis|cricket|pickleball|tabletennis), --video-path (required: file path),
--annotations-path (required: .json or .csv), --match-name (str, optional), --video-id (str,
optional), --license (str, default='Proprietary'), --fps (float, default=30.0), --resolution
(str, default='1080p')",
        "CSV input columns (required): rally_number (int or auto-indexed), start_time (float,
decimal seconds), end_time (float, decimal seconds); optional: score_before, score_after,
shots_count, ending_reason, last_shot_outcome. Case-insensitive header matching via lowercase
normalization.",
        "JSON input structure: list of objects, each with keys: rally_number (int), start_time
(float), end_time (float), plus optional fields matching CSV",
        "Badminton rally object contract: video_id (str), rally_number (int), start_time (float),
end_time (float); start_time must be < end_time; rally_number serves as chronological sort key",
        "DB schema: badminton_rallies table (video_id, rally_number, start_time, end_time,
score_before, score_after, shots_count, ending_reason, last_shot_outcome) with PK=(video_id,
rally_number), FK on videos.video_id with ON DELETE CASCADE",
        "Export format: per-video CSV at <out_dir>/<sport>/<video_id>.csv with header [start,end]
and (start_time, end_time) decimal-second tuples in chronological order; manifest.json with
entries {video_id, sport, csv, local_path, video_url, match_name, fps, resolution, license_type,
is_commercial, status, num_segments}"
    ],
    "gaps_or_mismatches": [
        "Fractional rally_number support: rally-annotator can produce decimal rally_number (e.g.,
2.5 for resegmented rallies); sports-data-collector expects int, converts via safe_int() which
truncates (2.5 -> 2), causing loss of distinction between fractional variants; no explicit error
on truncation",
        "Partial coverage handling: rally-annotator CSV may have gaps (e.g., rally 1,2,4 missing
3) or float numbers (1.1, 1.2); validator checks only temporal non-overlap and chronological
order via rally_number sort, not contiguity or valid numbering schemes",
        "Missing optional fields: sport column in rally-annotator CSV is self-documenting but
ignored by import-local; no validation that CSV sport matches --sport flag",
        "Score/outcome fields: rally-annotator tracks ending_reason enum
(winner|forced_error|unforced_error|service_fault|let|other); maps exactly to
BadmintonRally.ending_reason (optional); no score_before/score_after in rally-annotator, must be
None in import",
        "Null/empty handling: safe_float_required() treats empty string as ValueError; optional
int fields default to None (not 0) via safe_int(..., None) if missing; no distinction between
truly-missing and user-entered null",
        "Time format flexibility: parser supports only decimal seconds (float); does NOT support
mm:ss or HH:MM:SS time strings (unlike some consuming tools), only raw float values"
    ],
    "files": [
        "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
        "C:/Users/avidu/Projects/sports-data-collector/src/cli/ingest.py",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/parsing.py",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/badminton.py",
        "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md"
    ]
}

```

```

    ]
},
{
    "subsystem": "Canonical Rally Schema & CSV I/O (sports-data-collector)",
    "summary": "The sports-data-collector defines a canonical rally record schema
(BadmintonRally/PickleballRally/TableTennisRally) with required fields video_id, rally_number,
start_time, end_time, and optional fields score_before/after, shots_count, ending_reason,
last_shot_outcome. Validation enforces non-negative start times, strictly positive durations,
chronological ordering, and no temporal overlaps. The canonical CSV (ROORA_BRIEF format)
includes 11 columns: rally_number, start_mmss, end_mmss, start_time, end_time, shots_count,
ending_reason, last_shot_outcome, score_before, score_after, notes. CVAT XML is converted to
this canonical format; import-local CSV ingest and export to eval sets both read/write canonical
columns only, dropping all other fields.",
    "key_facts": [
        "BadmintonRally schema: C:/Users/avidu/Projects/sports-data-
collector/src/core/schemas/badminton.py:4-22. Required: video_id (str), rally_number (int),
start_time (float), end_time (float). Optional: score_before/score_after (str), shots_count
(int), ending_reason (str), last_shot_outcome (str).",
        "Validation: C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py:29-57
(validate_badminton_rallies). Rules: start_time >= 0, end_time > start_time
(Pydantic+validator), no temporal overlap between consecutive rallies, sorted by rally_number.",
        "Canonical CSV read: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:424-460 (load_canonical_csv). Reads decimal
start_time/end_time or mm:ss start_mmss/end_mmss; normalizes headers to lowercase; skips rows
missing rally_number or usable times; frame_min/frame_max set to -1 (CSV has no frame info).",
        "Canonical CSV write: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:356-372 (write_canonical_csv). 11 columns:
rally_number, start_mmss, end_mmss, start_time (%.3f), end_time (%.3f), shots_count,
ending_reason, last_shot_outcome, score_before, score_after, notes. Empty fields written as
blank strings.",
        "Canonical CSV columns (ROORA_BRIEF): C:/Users/avidu/Projects/sports-data-
collector/docs/annotation/ROORA_BRIEF.md:18-30. Enum ending_reason: 'winner' | 'forced_error' |
'unforced_error' | 'service_fault' | 'let' | 'other'. Enum last_shot_outcome: 'in' | 'net' |
'out' | 'n_a' (n_a for service_fault/let).",
        "Import-local CSV parse: C:/Users/avidu/Projects/sports-data-
collector/src/cli/curate.py:232-250. Reads CSV with DictReader, normalizes headers lowercase,
uses safe_int/safe_float for optional fields (defaults to None), required fields
(start_time/end_time) raise ValueError if missing. No sport/status/maturity/fractional columns
read; only 9 fields map to BadmintonRally.",
        "Export eval set: C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:440-536.
Writes per-video CSV with 2 columns: start, end (decimal seconds only). No rally_number,
ending_reason, last_shot_outcome, shots_count, or scores in export; uses get_segment_intervals
(start_time, end_time only from database).",
        "CVAT conversion: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:325-353 (convert). Time recomputed from
frame_min/frame_max: start_time = frame_min / fps, end_time = frame_max / fps (vendor
start_time/end_time attributes discarded). Issues reported (not auto-fixed): zero_duration,
ending_reason_vocab, outcome_vocab, shot_density, frame_out_of_range, overlap,
possible_missed_rally.",
        "Issue detection allowed vocabularies: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:50-61. DEFAULT_ALLOWED_REASONS: {'winner',
'forced_error', 'unforced_error', 'service_fault', 'let', 'other'}. DEFAULT_ALLOWED_OUTCOMES:
{'in', 'net', 'out', 'n_a'}.",
        "Database schema: C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:74-88.
badminton_rallies table: video_id (TEXT), rally_number (INTEGER), start_time (REAL), end_time
(REAL), score_before (TEXT), score_after (TEXT), shots_count (INTEGER), ending_reason (TEXT),
last_shot_outcome (TEXT). PK: (video_id, rally_number).",
        "Canonical column set on read (CSV / CVAT): rally_number, start_time (or start_mmss),
end_time (or end_mmss), shots_count, ending_reason, last_shot_outcome, score_before,
score_after, notes. Headers normalized to lowercase; missing optional fields degrade to None.",
        "Canonical column set on write (CSV export): ONLY start, end (decimal seconds). Rally
detail, ending_reason, shots_count, outcomes, scores, and video metadata NOT exported. Separate
manifest.json carries video_id, sport, num_segments, local_path, fps, resolution, license,
status.",
        "Canonical CSV in curate.py: C:/Users/avidu/Projects/sports-data-

```

```

collector/src/cli/curate.py:559 (write_canonical_csv arg). Called by convert-cvat command;
output CSV imports via import-local.",
    "Vendor comparison: C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/eval/eval_partial_labels.py (partial labels harness) and
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py:34-49
(load_golden_gts). Both read CSV with start_time, end_time columns ONLY; case-insensitive header
matching; no requirement for rally_number, ending_reason, shots_count, scores, or outcome
fields."
],
"contracts": [
    "BadmintonRally (& PickleballRally, TableTennisRally): video_id: str (required),
rally_number: int (required, ?1, chronological), start_time: float (required, ?0), end_time:
float (required, >start_time), score_before: str | None (default None), score_after: str | None
(default None), shots_count: int | None (default None), ending_reason: str | None (default
None), last_shot_outcome: str | None (default None). Pydantic validation: start_time ? 0;
end_time > start_time.",
    "Canonical CSV columns (on read from CVAT/import-local): rally_number (int, required),
start_time (float, required | start_mmss fallback), end_time (float, required | end_mmss
fallback), shots_count (int | None), ending_reason (str | None, vocab:
winner/forced_error/unforced_error/service_fault/let/other), last_shot_outcome (str | None,
vocab: in/net/out/n_a), score_before (str | None, format: 'A-B'), score_after (str | None,
format: 'A-B'), notes (str | None). Header normalization: all headers lowercased and stripped;
field matching is lenient (aliases for last_shot_outcome: outcome, last_shot_outcome
(in/net/out)).",
    "Canonical CSV columns (on write to export CSV): ONLY start (decimal seconds, float), end
(decimal seconds, float). NO rally_number, ending_reason, shots_count, scores, or outcome.
Written as 2-column CSV with header row.",
    "Canonical CSV columns (on write to CVAT-converted CSV): 11 columns in order:
rally_number, start_mmss (format m:ss.mmm), end_mmss, start_time (format %.3f), end_time (format
%.3f), shots_count (empty string if None), ending_reason (empty if None), last_shot_outcome
(empty if None), score_before (empty if None), score_after (empty if None), notes (empty if
None).",
    "Validation (validate_badminton_rallies): Returns (bool, List[str] errors). Checks:
start_time ? 0, end_time > start_time, sorted chronologically by rally_number, no temporal
overlap (end[i] <= start[i+1] for consecutive rallies). Blocker errors: timestamp violations,
overlaps. Both CVAT and CSV paths feed this validator.",
    "CVAT issue detection (detect_issues): Returns List[Issue] with severity ('blocker' |
'high' | 'medium' | 'low'), code (zero_duration, ending_reason_vocab, outcome_vocab,
shot_density, frame_out_of_range, overlap, possible_missed_rally), message (human-readable),
rally_number (optional). Blockers: zero_duration, overlap. NOT: sampling density (step/fps),
since downstream padding absorbs ~0.5s uncertainty.",
    "Ending reason vocabulary (canonical enum): {'winner', 'forced_error', 'unforced_error',
'service_fault', 'let', 'other'}. Shared across badminton, tennis, table tennis, pickleball,
padel. Last-shot-outcome is SEPARATE and orthogonal (winner can have outcome='in'; error can
have outcome='net'/'out'; service_fault/let have outcome='n_a').",
    "Import-local CSV format: flexible headers (case-insensitive), supports
start_mmss/end_mmss OR start_time/end_time (mm:ss parsed via formula or direct float).
rally_number defaults to 1-based row index if missing. shots_count, ending_reason,
last_shot_outcome, score_before, score_after, notes are all optional (degrade to None). No
sport/status/maturity/curation-level fields read; those come from CLI args."
],
"gaps_or_mismatches": [
    "ending_reason is PRESENT in canonical schema but NOT in eval CSV export (badminton-
highlight-indexer loads only start_time/end_time from golden CSVs). Implication: the rally-
indexer evaluation harness does NOT use ending_reason for scoring; it treats rallies as
unlabeled intervals.",
    "last_shot_outcome is PRESENT in canonical schema but NOT in eval CSV export (same as
ending_reason). The indexer calibration does not distinguish rally outcomes.",
    "shots_count is OPTIONAL in canonical schema and in CSV import but NOT captured in export
CSVs. Database stores it; export drops it. Unclear if downstream uses this field.",
    "Fractional rally numbers (e.g., 1.1, 1.2 for replays/lets) are NOT supported by the
canonical schema (rally_number is int). ROORA-BRIEF mandates integer 1-based rally_number with
no gaps. If vendors annotate sub-rallies (e.g., 1st serve fault = 1, 1st let = 1.1, 2nd serve =
1.2), the integer constraint will reject them or require manual splitting.",
    "Curation status / maturity metadata (e.g., STAGING vs CURATED, confidence flags, rework

```

needed) are NOT in the canonical CSV. They are stored in the database (videos.status enum: staging/curated/rejected) but do NOT round-trip through CSV export. Annotator/vendor CSVs have no analogous columns.",

"Missing end\_time is NOT explicitly handled by the canonical schema. ROORA\_BRIEF mandates end\_time > start\_time always. If a vendor omits end\_time or provides end\_time <= start\_time, load\_canonical\_csv skips the row silently (no error logged). Import-local raises ValueError if end\_time is missing/unparseable.",

"Annotation source / vendor / annotator ID is NOT in the canonical CSV. Video metadata carries match\_name and video\_id but no annotator/rework-status/version tracking per rally.",

"The canonical CSV is NARROWER on export than on import: export loses ending\_reason, last\_shot\_outcome, shots\_count, and scores. A CSV exported from the database and re-imported would preserve only start\_time/end\_time; all enrichment is lost. This is by design (eval set is intervals-only) but breaks round-trip fidelity.",

"CVAT attribute aliases are LENIENT (ending\_reason, outcome, last\_shot\_outcome (in/net/out) all map to canonical last\_shot\_outcome), but CSV import/export do NOT tolerate variant column names. If a vendor-submitted CSV has 'outcome' instead of 'last\_shot\_outcome', it is silently ignored.",

"Frame-based metadata (frame\_min, frame\_max, n\_boxes in CVAT RallyRecord) is NOT preserved in canonical CSV. Once converted to decimal seconds, frame info is lost.

RallyRecord.load\_canonical\_csv sets frame\_min/max to -1 as a marker."

```
],
"files": [
  "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/badminton.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/pickleball.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/tabletennis.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/parsing.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/parsers/badminton/coachai.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
  "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ROORA_BRIEF.md",
  "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py",
  "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/eval_partial_labels.py",
  "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md",
  "C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md"
]
```

```
{
  "subsystem": "Sports Data Collector ? CVAT Vendor Ingestion & Pipeline Role",
  "summary": "The collector is the data producer in a two-repo training pipeline. It curates sports videos with per-rally temporal annotations [start, end) and commercial-license metadata, then exports to the badminton-highlight-indexer for processing and offline rally-segmenter training. The Roora vendor ingestion flow converts CVAT XML exports (or canonical CSVs) through a four-command pipeline: convert-cvat (normalizing frame?seconds), validate-delivery (QA gate with blocker checks + human review sheet), import-local (SQLite registration), and export (indexer-ready CSV+manifest). Manual self-rating via the VLC rally-annotator tool provides an alternative, lightweight ingestion path with minimal metadata. Golden-set role: temporal-only annotations for segmentation evaluation and training, explicitly not for frame-level shuttle detection; non-commercial sources excluded from default exports.",
  "key_facts": [
```

"Collector role (PIPELINE\_ROLE.md:1-46): data producer feeding badminton-highlight-indexer; goal is offline rally segmenter trained on commercially-licensed, annotated corpus to replace Gemini API",

"Roora vendor pipeline (INGESTION\_PIPELINE.md:7-44): four sequential commands: convert-cvat ? validate-delivery ? import-local ? export; converts CVAT XML boxes+attributes to canonical CSV with frame?seconds recomputation",

"Critical frame-to-time conversion (rally\_cvat.py:24-29): recomputes from frame\_min/frame\_max using true fps; vendor's start\_time/end\_time attributes explicitly ignored to avoid systematic errors from step-sampled jobs with wrong fps assumptions",

"Canonical CSV schema (ROORA\_BRIEF.md:14-30): vendor provides rally\_number, start\_mmss, end\_mmss, shots\_count, ending\_reason, last\_shot\_outcome, score\_before/after, notes; collector auto-fills start\_time/end\_time as decimal seconds",

"Validation rules (INGESTION\_PIPELINE.md:68-92): blocker issues (overlaps, zero-duration)

gate the import; high/medium issues (shot\_density, possible\_missed\_rally, vocab) populate human review sheet; missed-rally detection via gap heuristic (>15s) and inflated-span flagging",

"Intake manifest fields (INTAKE\_MANIFEST\_TEMPLATE.csv:1-5): video\_id, sport, format, fps (critical), duration\_mmss, expected\_rallies, assigned\_to, status; fps provided to vendor for frame?seconds formula",

"VLC rally-annotator CSV (rally-annotator/README.md:10-14): minimal 5-column format (rally\_number, start\_time, end\_time, ending\_reason, sport) with decimal seconds only; no shots\_count/scores; resumable (continues numbering on re-open)",

"Rally-annotator ingestion (CSV\_FORMAT.md:26-41): direct import via curate import-local; optional fields populated as None; validation still runs (overlaps, positive duration); no external QA gate (self-produced)",

"Quality bar (ROORA\_BRIEF.md:79?86):  $\pm 0.3s$  boundary tolerance, zero missed rallies, schema compliance, shots\_count exact for ?15 shots or  $\pm 1$  for longer rallies, 20% human spot-check on every delivery",

"Golden-set temporal-only contract (ADR-002, GOLDEN\_SET\_IMPLEMENTATION.md:6-8): (video\_id, ordinal, start\_time, end\_time) shape shared across five sports; NOT used for shuttle detector training (lacks per-frame x,y labels); only for temporal segmenter training + eval",

"Regression testing (GS-2, GOLDEN\_SET\_IMPLEMENTATION.md:51-52): regress(video, tier, tolerance) compares predictions vs GT, flags when precision/recall drop >5% default; baseline snapshots per video with re-snapshot path for legitimate improvements",

"Non-commercial exclusion (INGESTION\_PIPELINE.md:124-129): videos kept in DB regardless of license; default export commercial-only; non-commercial sources excluded unless --include-noncommercial flag passed",

"Observability reports (INGESTION\_PIPELINE.md:131-144): per-video JSON (machine-readable), technical .md, and customer summary .md emitted on every validate-delivery/convert-cvat/import-local run for debugging/transparency"

],

"contracts": [

"Video identity contract: indexer keys by file MD5; collector by human video\_id; manifest carries exact local\_path so downstream processing runs on that exact file (acquisition precedes processing)",

"CSV schema contract: collector exports per-video (start, end) CSVs matching indexer's eval format; manifest.json pairs each CSV with video file; sport-agnostic (start\_time, end\_time) shape across badminton, tennis, cricket, pickleball, tabletennis",

"Canonical CSV format: rally\_number (1-based, no gaps), start\_time/end\_time (decimal seconds), shots\_count (optional int), ending\_reason (enum: winner/forced\_error/unforced\_error/service\_fault/let/other), last\_shot\_outcome (enum: in/net/out/n\_a), score\_before/after (optional 'A-B'), notes (required when ending\_reason=other)",

"CVAT attribute aliases: lenient matching for rally metadata (e.g. 'score\_before (a-b)' ? 'score\_before'); frame-derived times override vendor-supplied times; frame range [frame\_min, frame\_max] at true fps = [start\_time, end\_time] seconds",

"Validation blockers: overlaps (end[N] > start[N+1]), zero-duration (end ? start), frame\_out\_of\_range; violations exit non-zero and block import; non-blocker issues populate review sheet for human verification",

"Rally-annotator CSV format: rally\_number, start\_time (decimal), end\_time (decimal), ending\_reason, sport; optional columns may be added; consumers read by header name and skip rows where end ? start",

"Commercial license gate: is\_commercial flag per video; default export includes only commercial sources; non-commercial explicitly excluded unless --include-noncommercial passed",

"Temporal-only annotation contract: (video\_id, ordinal, start\_time, end\_time) shared across sports; no per-frame (x, y) shuttle coordinates; trained for segmentation (active vs. dead play), not detection; licensed/owned sources only"

],

"gaps\_or\_mismatches": [

"Partial-label strategy not explicitly documented: collector emphasizes 'zero missed rallies' as non-negotiable but offers no formal framework for handling genuinely ambiguous clips (warm-up detection, occluded play, administrative stoppages marked 'other')",

"Rally-annotator metadata subset: VLC tool omits shots\_count, scores, notes ? fields that vendor includes; no documented pathway to enrich rally-annotator CSVs with post-hoc shot counts or scores before/after import",

"Missed-rally detection limitations acknowledged but not solved: gap heuristic (>15s) and shot\_density flags are heuristic only; motion-based second-opinion stated as 'future enhancement' with no timeline",

"Quality bar vs. rally-annotator: vendor delivery expects 20% human spot-check; self-

produced rally-annotator has no equivalent QA gate (rater is both producer and verifier, single POV)",

"Warm-up/knock-up exclusion rule: golden rule in ANNOTATION\_GUIDE.md but no automated detection; rater must visually distinguish real play from pre-match cooperative rallying (relies on human judgment)",

"Frame sampling tolerance: documentation acknowledges ~0.5s sampling slack absorbed by clip padding but doesn't quantify impact on boundary accuracy for edge-case rallies",

"ADR-002 (golden set temporal-only) doesn't address: whether multi-annotator IAA (inter-annotator agreement) will be tracked; single-rater bias not factored into regression tolerance (ADR-004 reuses existing harness, may need sport-specific thresholds)"

```
],
"files": [
  "C:/Users/avidu/Projects/sports-data-collector/docs/PIPELINE_ROLE.md",
  "C:/Users/avidu/Projects/sports-data-collector/docs/INGESTION_PIPELINE.md",
  "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
  "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ROORA_BRIEF.md",
  "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ANNOTATION_GUIDE.md",
  "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv",
  "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
  "C:/Users/avidu/Projects/rally-annotator/README.md",
  "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md",
  "C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md",
  "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/GOLDEN_SET_IMPLEMENTATION.md",
  "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md"
]
},
{
```

"subsystem": "Manual Rally Annotation Artifacts",

"summary": "Two distinct CSV formats exist for manual rally annotation: (1) the lean rally-annotator output with 5 core columns (rally\_number, start\_time, end\_time, ending\_reason, sport), and (2) the richer rally\_review format with 14 columns adding metadata for manual curation (shot counts, time-in-minutes-seconds format, verification flags, reviewer comments). The ending\_reason taxonomy is shared across both: winner, forced\_error, unforced\_error, service\_fault, let, other. Manual ingesters must handle partial labeling (videos only partially tagged), fractional rally numbers (7.5, 16.5 from corrections), missing end\_time values, status flags (VERIFIED/needs review), and free-text notes capturing net touchdowns and flagged anomalies like OVERLAP or INFLATED timing.",

"key\_facts": [

"rally-annotator CSV columns (5): rally\_number (int, 1-based monotonic), start\_time (float, decimal seconds), end\_time (float, decimal seconds, always > start\_time), ending\_reason (enum: winner|forced\_error|unforced\_error|service\_fault|let|other), sport (enum: badminton|tennis|table\_tennis|pickleball|padel) - CSV\_FORMAT.md:6-18",

"rally\_review CSV columns (14): rally (int, rally identifier, may be fractional like 7.5 or 16.5), start\_mmss (string, mm:ss.sss format, e.g. '1:02.062'), end\_mmss (string, mm:ss.sss format), dur\_s (float, duration in seconds), shots (int, shot count for the rally), ending\_reason (enum or pipe-delimited alternates like 'forced\_error|other'), ending\_consistent (yes|NO, verification flag), notes (free-text, e.g. 'net', OVERLAP, INFLATED), auto\_flag (status field for automated review), VERIFIED(Y/N) (Y/N or empty), true\_start\_mmss (optional corrected start), true\_end\_mmss (optional corrected end), true\_ending (optional corrected reason), reviewer\_comment (optional notes) - rally\_review.csv header",

"Ending reason taxonomy (6 values, shared across sports): winner (opponent couldn't return in-court shot), forced\_error (loser missed under pressure), unforced\_error (loser missed routine shot with time+position), service\_fault (serve-rule violation), let (rally replayed under rules), other (occluded/administrative/unclear) - ENDING\_REASONS.md:7-9",

"rally\_annotator.lua line 88 hard-codes header:

'rally\_number,start\_time,end\_time,ending\_reason,sport\\n',

"rally\_annotator.lua line 262 writes CSV with %.3f precision for floats: 'd,%.3f,%.3f,%s,%s' (e.g. '1,37.434,72.194,winner,badminton')",

"Actual lean CSV (Adarsh and Avi.rallies.csv): 40 rallies, rally\_number 1-40 sequential, start\_time range 37.434-599.712s, end\_time range 72.194-604.962s, all have winning/error/service\_fault labels, all sport=badminton",

"Actual rich CSV (rally\_review.csv): 26 rows, rally column shows integers AND fractional numbers (1,2,3... up to 26 but stored as rally NOT rally\_number), times in mm:ss.sss format (e.g. '0:03.003', '2:29.649'), shot counts range 1-15, ending\_reason mostly single values but

row 16 shows 'forced\_error|other' (pipe-delimited alternatives), VERIFIED column has Y, NO, or empty, notes field captures qualitative flags ('net', 'OVERLAP', 'INFLATED'))",

"Manual ingester realities: (1) PARTIAL LABELING - video may be only partially tagged, labels cover only a prefix/window of total runtime, (2) FRACTIONAL RALLY NUMBERS - corrections/insertions create numbers like 7.5 or 16.5, not just integers, (3) MISSING end\_time - some fields may be blank (not shown in sample but possible), (4) STATUS FLAGS - VERIFIED(Y/N), ending\_consistent(yes/NO), auto\_flag field mark review state, (5) last\_shot\_outcome implicit in ending\_reason enum, (6) shots\_count as distinct column in review format, (7) FREE-TEXT NOTES - unstructured annotations like 'net touchdown' or anomaly flags ('INFLATED timing window', 'OVERLAP with prior rally')",

"rally\_review CSV uses time-in-minutes-seconds string format (mm:ss.sss) while rally-annotator uses decimal-seconds floats (crucial for parsing)",

"rally-annotator v1.3 supports 5 sports: badminton, tennis, table\_tennis, pickleball, padel (line 85-87 of Lua)",

"CSV parsing logic in rally\_annotator.lua lines 222-250: forgiving parser that preserves extra columns, treats any leading-integer line as a rally row, skips header and blank lines"

],

"contracts": [

"LEAN ANNOTATOR CSV (5 columns): rally\_number (int, 1-based, monotonic, continues across reopens), start\_time (float, decimal seconds), end\_time (float, decimal seconds, > start\_time), ending\_reason (enum: winner|forced\_error|unforced\_error|service\_fault|let|other), sport (enum: badminton|tennis|table\_tennis|pickleball|padel)",

"RICH REVIEW CSV (14 columns): rally (int or float, rally identifier, may be fractional), start\_mmss (string, mm:ss.sss format), end\_mmss (string, mm:ss.sss format), dur\_s (float, seconds), shots (int, shot count), ending\_reason (string, may be pipe-delimited alternatives like 'forced\_error|other'), ending\_consistent (enum: yes|NO), notes (string, free-text), auto\_flag (string, status field), VERIFIED(Y/N) (enum: Y|N|empty), true\_start\_mmss (string or empty), true\_end\_mmss (string or empty), true\_ending (string or empty), reviewer\_comment (string or empty)",

"ENDING\_REASON TAXONOMY (6 shared values across all sports): winner, forced\_error, unforced\_error, service\_fault, let, other",

"SPORT TAXONOMY (5 supported): badminton, tennis, table\_tennis, pickleball, padel",

"Decimal seconds format in lean CSV: floats with arbitrary precision but typically 3 decimal places (millisecond-level). Example: 37.434, 72.194",

"Time format in review CSV: string representation mm:ss.sss (minutes:seconds.milliseconds). Example: '0:03.003', '4:22.762'. Parser must convert to seconds for comparison with annotator output",

"Rally number uniqueness: within one video's CSV, rally\_number is unique and monotonic. Fractional numbers (7.5, 16.5) may appear in review CSV when corrections create inserted records"

],

"gaps\_or\_mismatches": [

"TIME FORMAT MISMATCH: rally-annotator outputs decimal-seconds floats (37.434); rally\_review uses mm:ss.sss string format (0:03.003). Ingester must convert review times to decimal-seconds or vice versa for reconciliation",

"COLUMN NAME MISMATCH: lean CSV has 'rally\_number' (int counter), but rich CSV uses 'rally' (may be fractional). Manual ingester must handle both naming conventions and account for fractional IDs from edits/corrections",

"MISSING end\_time: while both sample CSVs show complete (start, end) pairs, the schema allows blank end\_time. Ingester must validate end > start and reject incomplete rallies",

"FRACTIONAL RALLY NUMBERS: rally-annotator logic assumes 1-based integer sequencing (line 283-286 of Lua), but real-world manual edits/insertions in review CSV introduce 7.5, 16.5-style identifiers. Ingester must parse floats, not just integers",

"ENDING\_REASON ALTERNATIVES: lean CSV always single enum value; rich CSV row 16 shows 'forced\_error|other' (pipe-delimited), indicating human uncertainty or net-touched ambiguity. Ingester must handle both scalar and pipe-delimited alternatives",

"OPTIONAL CORRECTION COLUMNS: rich CSV has true\_start\_mmss, true\_end\_mmss, true\_ending columns (all empty in sample but semantically present). Ingester must treat these as optional overrides for the original times/reason if populated",

"PARTIAL LABELING: neither sample shows a video labeled only to 50% completion, but ENDING\_REASONS.md and Lua code (lines 438-464) reference the pattern. Ingester must accept incomplete, prefix-window labels (e.g. rallies 1-15 out of 50 total)",

"STATUS FLAG SEMANTICS: VERIFIED and ending\_consistent use inconsistent enum values (Y/N, yes/NO capitalization varies). Ingester must normalize or handle case-insensitively",

"FREE-TEXT NOTES CONTENT: notes field captures sport-specific edge cases ('net touchdown'



```

in badminton, 'wall bounce' for padel) without schema. Ingestor must preserve as-is or define a
taxonomy if automated handling is needed",
    "AUTO_FLAG AND REVIEW STATE: auto_flag field (empty in sample) indicates automated quality
checks triggered (e.g. OVERLAP, INFLATED). Ingestor must define what flags are actionable vs.
informational"
],
"files": [
    "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md",
    "C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md",
    "C:/Users/avidu/Projects/rally-annotator/vlc/rally_annotator.lua",
    "C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and Avi.rallies.csv",
    "C:/Users/avidu/Projects/Annotation Setup/rally_review.csv"
]
},
{
    "subsystem": "Collector Output Contract for Rally Indexer Eval/Training",
    "summary": "The indexer consumes rally ground truth from the collector via a **manifest.json
+ per-video CSV** contract. The manifest (created by `sports-data-collector`'s `curate export`
command) pairs each video with a CSV of decimal-second rally intervals, plus metadata (fps,
resolution, video path, sport, license, curation status). The CSV columns are minimal:
`start,end` in seconds. The indexer's eval and training pipelines use **temporal IoU matching**
(greedy one-to-one at a threshold, default 0.5) to compare predicted rally windows against
ground truth; partial labels are supported only in the dedicated `eval_partial_labels.py`
harness via explicit `--window-start`/`--window-end` flags, not represented in the export format
itself.",
    "key_facts": [
        "calibrate_wasb.py:34-49 / load_golden_gts() ? reads CSV with columns (case-insensitive):
start_time, end_time (decimal seconds); returns sorted list of (start, end) tuples",
        "calibrate_local.py:66-74 / build_videos() ? expects manifest JSON with structure [{name,
trajectory, fps, frame_width, labels}, ...]; labels = path to golden CSV",
        "eval_partial_labels.py:47-54 / restrict_to_window() ? implements labeled-window concept:
filter predictions/GT to [window_start, window_end] interval; predicted segments outside window
are dropped (neither rewarded nor penalized)",
        "eval_partial_labels.py:68-78 ? window_end defaults to max(end_time) of all GT rallies;
window_start defaults to 0.0; both are CLI args not part of CSV/manifest",
        "rally_slicer.py:53-73 / load_rally_labels() ? loads CSV with columns (case-insensitive):
rally_number, start_time, end_time (decimal seconds OR mm:ss/HH:MM:SS); returns dict
rally_number ? (start, end) seconds",
        "distill_local.py:81-87 / load_video() ? reads manifest entry {fps, frame_width,
trajectory, labels}; calls load_golden_gts(labels) to get GT intervals; label_candidates()
matches candidate windows to GT via IoU",
        "training.py:50-62 / label_candidates() ? uses match_intervals() with iou_threshold
(default 0.5); candidate is positive iff it matches exactly one GT interval at IoU >=
threshold",
        "metrics.py:48-79 / match_intervals() ? greedy one-to-one matching: highest-IoU pairs
claimed first, each pred/GT used at most once; only pairs with IoU >= iou_threshold are
considered for matching",
        "metrics.py:15-29 / interval_iou() ? temporal intersection-over-union: overlap/(union)
both measured as durations in seconds; returns 0.0 for non-overlapping or degenerate intervals",
        "sports-data-collector curate.py:496-509 / export_eval_set() ? manifest entry structure:
{video_id, sport, csv (relative path), local_path, video_url, match_name, fps, resolution,
license_type, is_commercial, status, num_segments}",
        "sports-data-collector curate.py:490-494 ? exports CSV with header 'start,end' and rows
[start_seconds, end_seconds] (decimal, computed from DB intervals); one rally per row",
        "annotations.py:44-76 / load_annotations() ? legacy simple format: reads CSV with optional
'start'/'start_time' header row; columns 1-2 are start/end timestamps (decimal OR colon-
separated MM:SS/HH:MM:SS); blank lines and # comments ignored",
        "calibrate_local.py:19-20 ? manifest expected from collector at: `../sports-data-
collector/data/eval_export/manifest.json` (relative path hint in docs)",
        "batch.py:22-40 / resolve_video_path() ? normalizes manifest local_path: strips './',
converts backslashes to forward slashes, joins with videos_root if relative; cross-platform
handling for Windows drive letters (C:/...)",
        "eval_partial_labels.py:72-82 ? labeled-window enforcement: load_golden_gts() gets all
rallies; manually compute/override window_end from CLI; restrict_to_window() filters both
predictions and GT to in-window region before scoring",

```

```

"distill_local.py:48-69 / build_candidates() ? proposal stage (velocity_thresh=0.005,
merge_gap=1.0, min_dur=0.5) generates many candidates; WASB trajectory parses trajectory.csv
with columns Frame, Visibility, X, Y"
],
"contracts": [
  "CSV (ground truth): columns start,end | format: decimal seconds (float) | one rally per
row | optional header 'start' or 'start_time' | blank/# lines ignored | no other columns
required/consumed",
  "CSV (human annotations, collector intake): columns rally_number, start_time, end_time,
ending_reason, sport [optional: start_mmss, end_mmss, shots_count, score_before, score_after,
last_shot_outcome, notes] | start/end can be decimal seconds OR MM:SS / HH:MM:SS | rally_number
= string (supports inserted IDs like '7.5') | indexer ignores all columns except
start_time/end_time",
  "manifest.json structure: {commercial_only, statuses, total_videos, total_segments,
eval_sets: [{video_id, sport, csv (relative path to start,end CSV), local_path, video_url,
match_name, fps (float), resolution (string or null), license_type, is_commercial (bool),
status, num_segments}, ...]}",
  "Trajectory CSV (WASB input): columns Frame, Visibility, X, Y | parsed by
TrackNetRunner.parse_trajectory_csv() | motion data required only for calibration/distillation,
not for evaluating labels alone",
  "IoU matching contract: temporal intervals (start, end) in seconds | IoU >= threshold ?
match candidate | greedy one-to-one ? TP counted | threshold defaults to 0.5, configurable via
--iou flag",
  "Labeled-window concept (partial labels): window_start, window_end (seconds) | predicted
intervals outside [window_start, window_end) are dropped before scoring | GT intervals assumed
already within window | NOT part of manifest/CSV; applied only in eval_partial_labels.py via CLI
args",
  "Metadata contract (in manifest): fps (float, required for trajectory windowing and
feature extraction) | frame_width (float, required for WASB velocity normalization) | local_path
(required, string, file system path; used as key with file MD5 for cross-repo identification) |
resolution (optional, string like '1080p')",
  "Feature extraction for training: candidate window bounds ? {duration, peak_velocity,
mean_velocity, active_frame_count (YOLO) OR active_ratio (WASB), track_id_count (YOLO) OR
net_crossings (WASB)} | features must be fps/resolution-INVARIANT except where explicitly
normalized by frame_width"
],
"gaps_or_mismatches": [
  "Labeled-window restriction (eval_partial_labels.py concept) is NOT representable in the
current collector export format ? it is enforced only as CLI args to eval_partial_labels.py, not
persisted in manifest/CSV. If collector needs to support partial labels at scale, a new optional
manifest field (e.g., window_start, window_end) would be required.",
  "Rally metadata (ending_reason, last_shot_outcome, score_before/after, shots_count) are
collected and validated by the collector but NOT consumed by indexer eval/training pipelines ?
they are discarded after validation. Only (start_time, end_time) flow downstream.",
  "Trajectory CSV path is NOT in the manifest ? it is supplied separately as a CLI arg
(--trajectory) to calibrate_wasb, calibrate_local, eval_partial_labels, and distill_local. The
distill pipeline requires it to build features but there is no contract guaranteeing it exists
or is co-located.",
  "Resolution field in manifest is optional (nullable) and rarely used downstream ? fps is
required but resolution is mainly for observability. Feature extraction normalizes by
frame_width instead of resolution string.",
  "Partial-label support assumes GT labels cover a PREFIX of the video only (every rally up
to window_end is labeled, everything after is unlabeled). This assumption is NOT validated or
documented in the manifest ? it is left to the user to enforce via --window-end.",
  "CSV header detection (start, start_time) is case-insensitive but relies on exact column
name matching (no fuzzy matching) ? a vendor sending 'start_seconds' or 'start_frame' would fail
silently or cause the harness to read the wrong column.",
  "No contract for multi-sport data in a single manifest ? eval_sets can contain mixed
sports but most eval/training scripts assume homogeneous sport (e.g., training.label_candidates
does not filter by sport, so mixing sports in one training set would train a sport-agnostic
model).",
  "IoU threshold is a global setting (--iou flag) applied uniformly across all videos in a
manifest ? no per-video or per-sport threshold override supported in the current contract."
],
"files": [

```

```

        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\calibrate_wasb.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\calibrate_local.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\eval_partial_labels.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\tools\\rally_slicer.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\distill_local.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\training.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\metrics.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\annotations.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\batch.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\run_phase3_eval.py",
        "C:\\Users\\avidu\\Projects\\sports-data-collector\\src\\cli\\curate.py"
    ]
},
{
    "subsystem": "Sports Data Collector - DB/Provenance & Observability Layer",
    "summary": "The collector stores video metadata and sport-specific annotations (rallies, points, deliveries) in SQLite with two-level schema: a central `videos` table holding provenance (source URL, license, sport, video_id) and curation status (STAGING/CURATED/REJECTED), plus sport-specific child tables (badminton_rallies, tennis_points, cricket_deliveries, etc.) containing segment metadata. Observability is layered via obsreport (soft dependency) which generates per-video reports at import/validation stages, capturing parse/validation/registration metrics and flags for delivery issues; local imports bypass vendor-issued flags but still emit reports showing license, commerciality, and segment counts at registration.",
    "key_facts": [
        "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:52-71: videos table schema ? video_id (PK), sport, license_type (enum: MIT/Apache/BSD/CC0/CC-BY/CC-BY-NC/CC-BY-NC-SA/Public-Domain/Proprietary/Unknown), license_url, is_commercial (boolean flag), status (enum: staging/curated/rejected), match_name, fps, resolution; foreign keys ensure on-delete cascade to child tables.",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py:24-27: CurationStatus enum ? STAGING, CURATED, REJECTED; used for video-level workflow state only, no segment-level maturity.",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py:12-22: LicenseType enum ? includes Proprietary (self-rated), MIT/Apache/BSD/CC0/CC-BY (vendor-delivered), CC-BY-NC/CC-BY-NC-SA (non-commercial), PUBLIC_DOMAIN, UNKNOWN; no source tracking (self-rated vs vendor-delivered not distinguished).",
        "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:109-122: import_local sets all local imports to CURATED status by default (line 117: 'Custom local imports bypass staging by default'), skipping vendor staging workflow.",
        "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:73-78: License commercial-safety check via validator.check_license_commercial(license_type) ? whitelist includes Proprietary (line 21 in validator.py), so self-rated Proprietary is treated as commercial-safe.",
        "C:/Users/avidu/Projects/sports-data-collector/src/reporting/adapters.py:113-156: build_import_recorder() ? observability envelope for import-local; parse stage logs records_parsed; validation stage reports is_valid (boolean); registration stage captures sport, license (value string), is_commercial, status (value string); no annotation author/timestamp/maturity.",
        "C:/Users/avidu/Projects/sports-data-collector/src/reporting/spec.yaml:25-47: Spec rules emit two flags at import: license_noncommercial (warn if non-commercial), missing_fps (warn if fps null). Rules only check video-level fields; no segment-level flags for imports.",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:210-237: insert_badminton_rallies() ? clears existing rallies for video_id then bulk-inserts new ones; no versioning, audit trail, or per-segment metadata beyond sport schema (rally_number, start_time, end_time, score, shots_count, ending_reason, last_shot_outcome).",
        "C:/Users/avidu/Projects/sports-data-collector/src/reporting/__init__.py:48-77: emit_import_report() ? called from curate.py:145-154 after DB insert; optional (soft dependency); writes next to video or to fallback_dir if video absent; always succeeds (never raises).",
        "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py: Issue class

```

? severity (blocker/high/medium/low), code, message, optional rally\_number; only delivery validation emits Issue objects; import-local does not detect or report Issues.",

"C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py:29-144:  
 validate\_\*\_rallies() ? checks temporal non-overlap and positive duration; no quality metrics, annotator tracking, or maturity scores.",

"C:/Users/avidu/Projects/sports-data-collector/tests/test\_reporting.py:83-112: import observability envelope captures only is\_valid (boolean), errors (list of strings), count (segments), sport, source\_format; no flags for import-local validation issues."

],

"contracts": [  
 "VideoMetadata schema: video\_id, sport(enum), video\_url?, local\_path?, license\_type(enum), license\_url?, is\_commercial(bool), status(CurationStatus enum: staging/curated/rejected), match\_name, fps?, resolution?",  
 "CurationStatus enum: STAGING | CURATED | REJECTED ? video-level only, no segment maturity states",  
 "LicenseType enum: MIT, Apache-2.0, BSD, CC0, CC-BY, CC-BY-NC, CC-BY-NC-SA, Public-Domain, Proprietary, Unknown ? no source marker (self-rated vs vendor-delivered)",  
 "is\_commercial: boolean derived from license\_type whitelist check; whitelist in config.yaml includes Proprietary (defaults to [MIT, Apache-2.0, BSD, CC0, CC-BY, Public-Domain, Proprietary])",  
 "Import observability envelope (obsreport.RunRecorder): stages=(parse, validation, registration); parse.metrics.records\_parsed; validation.status(ok/error), validation.metrics.valid(bool); registration.metrics.segments\_registered(int), registration.details={sport, license(str), is\_commercial(bool), status(str)}",  
 "Delivery observability envelope: stages=(parse, validation);  
 parse.metrics.rallies\_parsed; validation.metrics.issues\_blocker/high/medium/low; flags carry code, severity(error/warn/info), message, stage, customer\_message?, evidence={rally\_number?, rally\_frames?, fps?}",  
 "Spec rules: license\_noncommercial (warn if is\_commercial==false), missing\_fps (warn if fps==null); rules trigger only at import/delivery validation, never at re-tagging",  
 "RallyRecord (CVAT adapter): rally\_number, start\_time, end\_time, shots\_count?, ending\_reason?, last\_shot\_outcome?, score\_before?, score\_after?, notes?, frame\_min, frame\_max, n\_boxes, vendor\_start\_time?, vendor\_end\_time?",  
 "Database cascade: video DELETE triggers cascade on child tables (badminton\_rallies, tennis\_points, cricket\_deliveries, pickleball\_rallies, tabletennis\_rallies); child tables have no independent delete cascade"

],

"gaps\_or\_mismatches": [  
 "No annotation source tracking: LicenseType enum includes Proprietary but does not distinguish self-rated (manual local import) from vendor-delivered (CVAT/external parser); license provenance assumes external source unless registered via import-local, but no explicit marker.",  
 "No segment-level maturity: segments (rallies/points/deliveries) lack status/maturity fields; video-level STAGING/CURATED/REJECTED does not map to individual segment review state (needs\_review -> confirmed -> gold not trackable).",  
 "No annotator/author metadata: neither VideoMetadata nor rally schemas capture who created/reviewed annotations; no timestamp for import or re-tagging.",  
 "Local import skips staging workflow: import\_local (line 117 in curate.py) hardcodes CURATED status, bypassing vendor review pipeline; observability report does not flag this deviation.",  
 "Import validation does not emit observability flags:  
 DatasetValidator.validate\_\*\_rallies() only check temporal integrity (overlaps, duration); emit\_import\_report() logs errors only as string list, not structured Flags; no per-segment QA metrics.",  
 "No re-tagging audit trail: insert\_badminton\_rallies() (line 215-216 in db.py) DELETE-then-INSERT erases history when re-importing annotations for the same video\_id; no versioning or conflict detection.",  
 "No label quality regression tracking: no metrics for comparing old vs. new annotations when re-tagging (e.g., shot\_count changes, ending\_reason divergence); no drift detection.",  
 "License whitelist hardcoded in config fallback: check\_license\_commercial() (validator.py:21) defaults to fixed list including Proprietary; no per-video or per-sport override; no rationale/audit trail for why Proprietary is commercial-safe.",  
 "Observability rules only video-level: spec.yaml:25-47 defines only video-scope rules (license\_noncommercial, missing\_fps); no rules for segment defects at import time (gaps, outliers, schema violations).",



```

191         game_score=obj.get("game_score"),
192         server=obj.get("server"),
193         ending_type=obj.get("ending_type")
194     ))
195     elif sport == "cricket":
196         parsed_items.append(CricketDelivery(
197             video_id=video_id,
198             over=_safe_int(obj.get("over"), 0),
199             ball=_safe_int(obj.get("ball"), 0),
200             start_time=_safe_float(obj.get("start_time")),
201             end_time=_safe_float(obj.get("end_time")),
202             batsman=obj.get("batsman"),
203             bowler=obj.get("bowler"),
204             runs=_safe_int(obj.get("runs"), None),
205             wicket=bool(obj.get("wicket", False)),
206             extras=_safe_int(obj.get("extras"), 0)
207         ))
208     elif sport == "pickleball":
209         parsed_items.append(PickleballRally(
210             video_id=video_id,
211             rally_number=_safe_int(obj.get("rally_number"), idx + 1),
212             start_time=_safe_float(obj.get("start_time")),
213             end_time=_safe_float(obj.get("end_time")),
214             score_before=obj.get("score_before"),
215             score_after=obj.get("score_after"),
216             shots_count=_safe_int(obj.get("shots_count"), None),
217             ending_reason=obj.get("ending_reason"),
218             last_shot_outcome=obj.get("last_shot_outcome")
219         ))
220     elif sport == "tabletennis":
221         parsed_items.append(TableTennisRally(
222             video_id=video_id,
223             rally_number=_safe_int(obj.get("rally_number"), idx + 1),
224             start_time=_safe_float(obj.get("start_time")),
225             end_time=_safe_float(obj.get("end_time")),
226             score_before=obj.get("score_before"),
227             score_after=obj.get("score_after"),
228             shots_count=_safe_int(obj.get("shots_count"), None),
229             ending_reason=obj.get("ending_reason"),
230             last_shot_outcome=obj.get("last_shot_outcome")
231         ))
232     elif ext == '.csv':
233         with open(filepath, 'r', encoding='utf-8') as f:
234             reader = csv.DictReader(f)
235             # Normalize headers
236             reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []
237
238         for idx, row in enumerate(reader):
239             if sport == "badminton":
240                 parsed_items.append(BadmintonRally(
241                     video_id=video_id,
242                     rally_number=_safe_int(row.get("rally_number"), idx + 1),
243                     start_time=_safe_float(row.get("start_time")),
244                     end_time=_safe_float(row.get("end_time")),
245                     score_before=row.get("score_before"),
246                     score_after=row.get("score_after"),
247                     shots_count=_safe_int(row.get("shots_count"), None),
248                     ending_reason=row.get("ending_reason"),
249                     last_shot_outcome=row.get("last_shot_outcome")
250                 ))
251             elif sport == "tennis":
252                 parsed_items.append(TennisPoint(
253                     video_id=video_id,
254                     point_number=_safe_int(row.get("point_number"), idx + 1),

```

```

255         start_time=_safe_float(row.get("start_time")),
256         end_time=_safe_float(row.get("end_time")),
257         set_score=row.get("set_score"),
258         game_score=row.get("game_score"),
259         server=row.get("server"),
260         ending_type=row.get("ending_type")
261     ))
262     elif sport == "cricket":
263         parsed_items.append(CricketDelivery(
264             video_id=video_id,
265             over=_safe_int(row.get("over"), 0),
266             ball=_safe_int(row.get("ball"), 0),
267             start_time=_safe_float(row.get("start_time")),
268             end_time=_safe_float(row.get("end_time")),
269             batsman=row.get("batsman"),
270             bowler=row.get("bowler"),
271             runs=_safe_int(row.get("runs"), None),
272             wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
273             extras=_safe_int(row.get("extras"), 0)
274         ))
275     elif sport == "pickleball":
276         parsed_items.append(PickleballRally(
277             video_id=video_id,
278             rally_number=_safe_int(row.get("rally_number"), idx + 1),
279             start_time=_safe_float(row.get("start_time")),
280             end_time=_safe_float(row.get("end_time")),
281             score_before=row.get("score_before"),
282             score_after=row.get("score_after"),
283             shots_count=_safe_int(row.get("shots_count"), None),
284             ending_reason=row.get("ending_reason"),
285             last_shot_outcome=row.get("last_shot_outcome")
286         ))
287     elif sport == "tabletennis":
288         parsed_items.append(TableTennisRally(
289             video_id=video_id,
290             rally_number=_safe_int(row.get("rally_number"), idx + 1),
291             start_time=_safe_float(row.get("start_time")),
292             end_time=_safe_float(row.get("end_time")),
293             score_before=row.get("score_before"),
294             score_after=row.get("score_after"),
295             shots_count=_safe_int(row.get("shots_count"), None),
296             ending_reason=row.get("ending_reason"),
297             last_shot_outcome=row.get("last_shot_outcome")
298         ))
299     else:
300         raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
301
302     return parsed_items
303
304     def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
305         """Download one video at best resolution, retrying transient failures.

```

#### 4. user (2026-06-08T08:43:47.251Z)

```

1     # Output CSV format
2
3     One row per rally. Header is written once when the file is created.
4
5     ```
6     rally_number,start_time,end_time,ending_reason,sport
7     1,8.800,11.500,winner,badminton
8     2,24.389,46.589,unforced_error,badminton
9     3,49.183,54.683,let,badminton

```

```

10     ``
11
12     | Column | Type | Notes |
13     |---|---|---|
14     | `rally_number` | int | 1-based, monotonic; continues across re-opening the same CSV |
15     | `start_time` | float | rally start, **decimal seconds** from the video start |
16     | `end_time` | float | rally end, decimal seconds (always `> start_time`) |
17     | `ending_reason` | enum | `winner` / `forced_error` / `unforced_error` /
`service_fault` / `let` / `other` ? see the decision guide in
[ENDING_REASONS.md](ENDING_REASONS.md) |
18     | `sport` | enum | `badminton` / `tennis` / `table_tennis` / `pickleball` / `padel` |
19
20     Notes:
21     - **Decimal seconds**, not `mm:ss` ? downstream loaders parse floats.
22     - Rows are appended live; **Undo last** truncates the last row (one level).
23     - Consumers should read by **column name** (the header is stable but column *order* may
evolve; extra
24         columns may be added later). Most CSV `DictReader`-style loaders already tolerate
extra columns.
25
26     ## Ingesting the labels
27
28     This CSV is intentionally compatible with common rally-segmentation tooling that reads
29     `rally_number,start_time,end_time` by header name and drops rows where `end <= start`:
30
31     ```bash
32     # Generic: any tool that takes a labels CSV of start/end seconds.
33
34     # Example ? score a detector against these labels, or slice clips by them
35     # (badminton-highlight-indexer loaders read --labels <this.csv> directly):
36     python -m backend.eval.calibrate_wasb --trajectory traj.csv --labels match.rallies.csv
--fps 59.94 --frame-width 1920
37     python -m backend.tools.rally_slicer --video match.mp4 --labels match.rallies.csv
--rallies all
38
39     # Example ? register as curated golden data in sports-data-collector:
40     python -m src.cli.curate import-local --sport badminton --video-path match.mp4
--annotations-path match.rallies.csv
41     ```
42
43     The `sport` column is self-documenting; tools that take a `--sport` flag can ignore it.
44

```

## 5. assistant (2026-06-08T08:43:51.148Z)

Let me verify the helper functions and the rally-annotator's actual partial CSV, plus the indexer eval loader, to ground the partial-coverage and fractional-rally handling.

## 6. user (2026-06-08T08:43:52.747Z)

No matches found

## 7. user (2026-06-08T08:43:53.197Z)

```

52     def import_local(self, args):
53         """Manually registers a local video and annotation file into the database."""
54         sport_str = args.sport.lower()
55         if sport_str not in self.config.get("supported_sports", ["badminton"]):
56             print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
57             sys.exit(1)
58
59         # Validate video path
60         if not os.path.exists(args.video_path):

```



```

61         print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63         # Validate annotation file
64         if not os.path.exists(args.annotations_path):
65             print(f"Error: Annotations file '{args.annotations_path}' not found.")
66             sys.exit(1)
67
68         match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
69         video_id = args.video_id or match_name.lower().replace(" ", "_")
70
71         # Parse license
72         try:
73             license_type = LicenseType(args.license)
74         except ValueError:
75             print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76             license_type = LicenseType.UNKNOWN
77
78         is_commercial = self.validator.check_license_commercial(license_type)
79
80         # 1. Parse Annotations
81         annotations = []
82         try:
83             annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
84         except Exception as e:
85             print(f"Error parsing annotations: {e}")
86             sys.exit(1)
87
88         # 2. Run Validation
89         is_valid = True
90         errors = []
91         if sport_str == "badminton":
92             is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93         elif sport_str == "tennis":
94             is_valid, errors = self.validator.validate_tennis_points(annotations)
95         elif sport_str == "cricket":
96             is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97         elif sport_str == "pickleball":
98             is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99         elif sport_str == "tabletennis":
100             is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102         if not is_valid:
103             print(f"Error: Annotations failed validation!")
104             for err in errors:
105                 print(f" - {err}")
106             sys.exit(1)
107
108         # 3. Create Video Metadata
109         video_meta = VideoMetadata(
110             video_id=video_id,
111             sport=SportType(sport_str),
112             video_url=None,
113             local_path=args.video_path,
114             license_type=license_type,
115             license_url=None,
116             is_commercial=is_commercial,
117             status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
118             match_name=match_name,
119             fps=args.fps,
120             resolution=args.resolution
121         )

```

```

122
123         # 4. Insert into Database
124         self.db.insert_video(video_meta)
125
126         count = 0
127         if sport_str == "badminton":
128             count = self.db.insert_badminton_rallies(annotations)
129         elif sport_str == "tennis":
130             count = self.db.insert_tennis_points(annotations)
131         elif sport_str == "cricket":
132             count = self.db.insert_cricket_deliveries(annotations)
133         elif sport_str == "pickleball":
134             count = self.db.insert_pickleball_rallies(annotations)
135         elif sport_str == "tabletennis":
136             count = self.db.insert_tabletennis_rallies(annotations)
137
138         print(f"\nSuccessfully imported local video '{match_name}':")
139         print(f"  - Video ID: {video_id}")
140         print(f"  - Sport: {sport_str}")
141         print(f"  - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
142         print(f"  - Registered {count} annotated segments.\n")
143
144         # Per-video observability report (next to the video). Never raises.
145         from src.reporting import emit_import_report
146         src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
147         report_paths = emit_import_report(
148             video_meta_obj=video_meta, annotations=annotations, is_valid=True,
149             errors=[], count=count, sport=sport_str, source_format=src_fmt,
150             fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
151         )
152         if report_paths:
153             print(f"  - Observability report: {report_paths.get('technical_md')}")
154             print(f"  - Customer summary      : {report_paths.get('customer_md')}\n")
155

```

## 8. assistant (2026-06-08T08:43:56.826Z)

The ``_safe_int`/`_safe_float`` are module-level (findings reference ``safe_int`/`safe_float``). Let me find their exact definitions and the validator, then check the actual partial CSV and the indexer's eval loader to confirm what flows downstream.

## 9. user (2026-06-08T08:43:58.466Z)

No matches found

## 10. user (2026-06-08T08:43:59.213Z)

```

1     from typing import List, Tuple, Union
2     from src.core.config import load_config
3     from src.core.schemas.base import VideoMetadata, LicenseType
4     from src.core.schemas.badminton import BadmintonRally
5     from src.core.schemas.tennis import TennisPoint
6     from src.core.schemas.cricket import CricketDelivery
7     from src.core.schemas.pickleball import PickleballRally
8     from src.core.schemas.tabletennis import TableTennisRally
9
10    class DatasetValidator:
11        def __init__(self, config_path: str = "config/config.yaml"):
12            self.config_path = config_path
13            self.commercial_licenses = self._load_licenses_whitelist()
14
15        def _load_licenses_whitelist(self) -> List[str]:

```

```

16         # load_config returns {} on a missing/unreadable file, so the default list
17         # below is the single fallback (keep in sync with config.yaml).
18         config = load_config(self.config_path)
19         return config.get(
20             'commercial_licenses',
21             ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-Domain",
"Proprietary"],
22         )
23
24     def check_license_commercial(self, license_type: Union[LicenseType, str]) -> bool:
25         """Determines if a license type is safe for commercial use based on the
configuration whitelist."""
26         val = license_type.value if isinstance(license_type, LicenseType) else
license_type
27         return val in self.commercial_licenses
28
29     def validate_badminton_rallies(self, rallies: List[BadmintonRally]) -> Tuple[bool,
List[str]]:
30         """
31         Validates badminton rallies:
32         - Sorted chronologically.
33         - Start time < End time (handled by Pydantic, but double checked).
34         - No temporal overlap between sequential rallies.
35         """
36         errors = []
37         if not rallies:
38             return True, errors
39
40         # Sort by rally number to check progression
41         sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
42
43         for i in range(len(sorted_rallies)):
44             r = sorted_rallies[i]
45             if r.start_time >= r.end_time:
46                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
47
48             if i > 0:
49                 prev = sorted_rallies[i - 1]
50                 if r.start_time < prev.end_time:
51                     errors.append(
52                         f"Rally overlap detected between Rally {prev.rally_number} "
53                         f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
54                         f"(starts at {r.start_time}s)"
55                     )
56
57         return len(errors) == 0, errors
58
59     def validate_tennis_points(self, points: List[TennisPoint]) -> Tuple[bool,
List[str]]:
60         errors = []
61         if not points:
62             return True, errors
63
64         sorted_pts = sorted(points, key=lambda p: p.point_number)
65         for i in range(len(sorted_pts)):
66             pt = sorted_pts[i]
67             if pt.start_time >= pt.end_time:
68                 errors.append(f"Point {pt.point_number}: start ({pt.start_time}) >= end
({pt.end_time})")
69
70             if i > 0:
71                 prev = sorted_pts[i - 1]
72                 if pt.start_time < prev.end_time:
73                     errors.append(

```

```

74             f"Point overlap detected: Point {prev.point_number} (ends
{prev.end_time}s) "
75             f"and Point {pt.point_number} (starts {pt.start_time}s)"
76         )
77         return len(errors) == 0, errors
78
79     def validate_cricket_deliveries(self, deliveries: List[CricketDelivery]) ->
Tuple[bool, List[str]]:
80         errors = []
81         if not deliveries:
82             return True, errors
83
84         # Sort by over, then ball within over
85         sorted_delivs = sorted(deliveries, key=lambda d: (d.over, d.ball))
86         for i in range(len(sorted_delivs)):
87             d = sorted_delivs[i]
88             if d.start_time >= d.end_time:
89                 errors.append(f"Over {d.over} Ball {d.ball}: start ({d.start_time}) >=
end ({d.end_time})")
90
91             if i > 0:
92                 prev = sorted_delivs[i - 1]
93                 if d.start_time < prev.end_time:
94                     errors.append(
95                         f"Delivery overlap: Over {prev.over} Ball {prev.ball} (ends
{prev.end_time}s) "
96                         f"and Over {d.over} Ball {d.ball} (starts {d.start_time}s)"
97                     )
98         return len(errors) == 0, errors
99
100    def validate_pickleball_rallies(self, rallies: List[PickleballRally]) -> Tuple[bool,
List[str]]:
101        """Validates pickleball rallies: positive durations and no temporal overlap
102        between chronologically-ordered rallies (same contract as badminton)."""
103        errors = []
104        if not rallies:
105            return True, errors
106
107        sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
108        for i in range(len(sorted_rallies)):
109            r = sorted_rallies[i]
110            if r.start_time >= r.end_time:
111                errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
112
113            if i > 0:
114                prev = sorted_rallies[i - 1]
115                if r.start_time < prev.end_time:
116                    errors.append(
117                        f"Rally overlap detected between Rally {prev.rally_number} "
118                        f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
119                        f"(starts at {r.start_time}s)"
120                    )
121        return len(errors) == 0, errors
122
123    def validate_tabletennis_rallies(self, rallies: List[TableTennisRally]) ->
Tuple[bool, List[str]]:
124        """Validates table-tennis rallies: positive durations and no temporal overlap
125        between chronologically-ordered rallies (same contract as badminton)."""
126        errors = []
127        if not rallies:
128            return True, errors
129
130        sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
131        for i in range(len(sorted_rallies)):

```

```

132         r = sorted_rallies[i]
133         if r.start_time >= r.end_time:
134             errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
135
136         if i > 0:
137             prev = sorted_rallies[i - 1]
138             if r.start_time < prev.end_time:
139                 errors.append(
140                     f"Rally overlap detected between Rally {prev.rally_number} "
141                     f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
142                     f"(starts at {r.start_time}s)"
143                 )
144         return len(errors) == 0, errors
145

```

#### 11. user (2026-06-08T08:43:59.288Z)

```

1     from pydantic import BaseModel, Field, model_validator
2     from typing import Optional
3
4     class BadmintonRally(BaseModel):
5         video_id: str
6         rally_number: int = Field(..., description="Chronological rally index in the match")
7         start_time: float = Field(..., description="Start of rally in seconds")
8         end_time: float = Field(..., description="End of rally in seconds")
9         score_before: Optional[str] = Field(None, description="Score before the rally (e.g.
'0-0')")
10        score_after: Optional[str] = Field(None, description="Score after the rally (e.g.
'1-0')")
11        shots_count: Optional[int] = Field(None, description="Number of strokes in the
rally")
12        ending_reason: Optional[str] = Field(None, description="Why the rally ended:
'winner' | 'forced_error' | 'unforced_error' | 'service_fault' | 'let' | 'other'")
13        last_shot_outcome: Optional[str] = Field(None, description="Where the last shot
landed: 'in' | 'net' | 'out' | 'n_a' (n_a for service_fault/let)")
14
15        @model_validator(mode='after')
16        def validate_timestamps(self) -> 'BadmintonRally':
17            if self.start_time < 0:
18                raise ValueError("start_time must be non-negative")
19            if self.end_time <= self.start_time:
20                raise ValueError("end_time must be strictly greater than start_time")
21            return self
22

```

#### 12. assistant (2026-06-08T08:44:03.460Z)

Now let me find the safe helpers' exact location and signature, plus confirm the actual partial CSV files and the indexer's eval loader.

#### 13. user (2026-06-08T08:44:07.347Z)

```

1     """Shared parsing helpers used by the CLI importer and dataset parsers.
2
3     Previously these were duplicated as private ``_safe_int`` / ``_safe_float`` /
4     ``_parse_time_string`` in both ``src/cli/curate.py`` and the CoachAI parser.
5     """
6     from typing import Any, Optional
7
8
9     def safe_int(value: Any, default: Optional[int] = 0) -> Optional[int]:
10         """Parse an int from a possibly-empty/float/None value; return default on
failure."""

```

```

11         if value is None:
12             return default
13         s = str(value).strip()
14         if not s:
15             return default
16         try:
17             return int(float(s))
18         except (TypeError, ValueError):
19             return default
20
21
22     def safe_float_required(value: Any) -> float:
23         """Parse a float; raise ValueError if missing/empty (caller treats it as
24         required)."""
25         if value is None:
26             raise ValueError("missing required numeric value")
27         s = str(value).strip()
28         if not s:
29             raise ValueError("missing required numeric value")
30         return float(s)
31
32     def parse_time_string(time_str: Any) -> float:
33         """Convert 'HH:MM:SS' / 'MM:SS' / float-strings to float seconds; 0.0 if
34         blank/invalid."""
35         if not time_str:
36             return 0.0
37         s = str(time_str).strip()
38         if ":" in s:
39             parts = s.split(":")
40             try:
41                 if len(parts) == 3:
42                     return float(parts[0]) * 3600 + float(parts[1]) * 60 + float(parts[2])
43                 if len(parts) == 2:
44                     return float(parts[0]) * 60 + float(parts[1])
45             except ValueError:
46                 return 0.0
47         try:
48             return float(s)
49         except ValueError:
50             return 0.0

```

#### 14. user (2026-06-08T08:44:07.411Z)

```

11:from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
175:         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
176:         start_time=_safe_float(obj.get("start_time")),
177:         end_time=_safe_float(obj.get("end_time")),
180:         shots_count=_safe_int(obj.get("shots_count"), None),
187:         point_number=_safe_int(obj.get("point_number"), idx + 1),

```

[Showing results with pagination = limit: 6]

#### 15. user (2026-06-08T08:44:07.429Z)

```

40     Config = Tuple[float, float, float]
41
42     # Operating points to compare. The tuned config wins on its fit-video but over-segments
43     # and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
44     TUNED: Config = (0.05, 1.0, 1.0)
45
46
47     def restrict_to_window(intervals: List[Interval], w0: float, w1: float) ->
48     List[Interval]:

```

```

48     """Keep intervals that overlap the labeled window [w0, w1].
49
50     A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
51     in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
52     they are neither rewarded nor penalized. GTs are assumed already inside the window.
53     """
54     return [(s, e) for (s, e) in intervals if e > w0 and s < w1]
55
56
57     def _eval_point(predict_fn, cfg: Config, points, fps: float, gts: List[Interval],
58                     window: Tuple[float, float], gate_crossings: int, iou: float) -> dict:
59         """Score one (config, gate) operating point inside the labeled window."""
60         preds = predict_fn(cfg)
61         if gate_crossings > 0 and points:
62             preds = rally_gate.apply_gate(points, preds, fps, min_crossings=gate_crossings)
63             preds = restrict_to_window(preds, *window)
64             sc = score(preds, gts, iou)
65             return {"cfg": cfg, "gate": gate_crossings, "n_pred": len(preds), "score": sc,
66 "preds": preds}
67
68     def run(trajecory_csv: str, labels_csv: str, fps: float, frame_width: float,
69            window_start: float = 0.0, window_end: Optional[float] = None,
70            gate_crossings: int = 2, iou: float = 0.5,
71            comparison_out: Optional[str] = None) -> dict:
72         gts_all = load_golden_gts(labels_csv)
73         if not gts_all:
74             raise SystemExit(f"no usable human rallies in {labels_csv}")
75         if window_end is None:
76             window_end = max(e for _, e in gts_all)
77         window = (window_start, window_end)
78         gts = restrict_to_window(gts_all, *window)
79
80         predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
81         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
82         traj_end = (max(p.frame for p in points) / fps) if points else 0.0
83
84         print("=== Pipeline vs PARTIAL human labels ===")
85         print(f"labels: {labels_csv}")
86         print(f" {len(gts_all)} human rallies; labeled window [{window_start:.1f}s,
87 {window_end:.1f}s] "
88             f"-> {len(gts)} rallies in-window")
89         print(f"trajectory covers ~0-{traj_end:.1f}s "
90             f"({'covers the window' if traj_end >= window_end - 1 else 'WARNING: shorter
than window'})")

```

## 16. assistant (2026-06-08T08:44:10.250Z)

Now I have the full picture. Let me confirm the actual partial CSV files exist and check the `export\_eval\_set` and `load\_golden\_gts` to confirm the export/consume contract.

## 17. user (2026-06-08T08:44:11.690Z)

```

30
31     BASELINE: Config = (0.02, 2.0, 2.0)    # the WasbConfig / trajectory_to_action_windows
defaults
32
33
34     def load_golden_gts(csv_path: str) -> List[Interval]:
35         """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
36         gts: List[Interval] = []
37         with open(csv_path, newline="", encoding="utf-8") as f:
38             reader = csv.DictReader(f)
39             reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[])]

```

```

40         for row in reader:
41             row = {(k or "").strip().lower(): v for k, v in row.items()}
42             try:
43                 s = float((row.get("start_time") or "").strip())
44                 e = float((row.get("end_time") or "").strip())
45             except (TypeError, ValueError):
46                 continue
47             if e > s:
48                 gts.append((s, e))
49         return sorted(gts)
50
51
52     def make_predict_fn(trajecory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
53         """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
54         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)

```

## 18. user (2026-06-08T08:44:13.130Z)

```

440     def export_eval_set(self, args):
441         """Export curated annotations as indexer-ready eval/training sets.
442
443         Emits, per qualifying video, a `//.csv>` in the
444         indexer's `start,end` ground-truth format (decimal seconds), plus a single
445         `/manifest.json` that pairs each CSV with its video file. The
446         manifest is the contract the downstream eval/training driver reads: the
447         indexer keys videos by *file MD5*, not by our `video_id`, so it must run
448         against the exact `local_path`/`video_url` recorded here.
449
450         By default only CURATED + commercially-safe videos are exported, mirroring
451         the license gate already enforced elsewhere in the toolchain.
452         """
453         out_dir = args.out_dir
454         commercial_only = not args.include_noncommercial
455         statuses = [CurationStatus.CURATED]
456         if args.include_staging:
457             statuses.append(CurationStatus.STAGING)
458
459         supported = self.config.get("supported_sports", ["badminton"])
460         sport_strs = [args.sport] if args.sport else supported
461
462         manifest = []
463         skipped_noncomm = 0
464         skipped_empty = 0
465
466         for sport_str in sport_strs:
467             try:
468                 sport = SportType(sport_str.lower())
469             except ValueError:
470                 print(f"Warning: skipping unknown sport '{sport_str}'.")
471                 continue
472
473             videos = []
474             for status in statuses:
475                 videos.extend(self.db.get_videos_by_status(status, sport))
476
477             for video in videos:
478                 if commercial_only and not video.is_commercial:
479                     skipped_noncomm += 1
480                     continue
481
482                 intervals = self.db.get_segment_intervals(video.video_id, sport)
483                 if not intervals:
484                     skipped_empty += 1
485                     continue

```



```

486
487     sport_dir = os.path.join(out_dir, sport.value)
488     os.makedirs(sport_dir, exist_ok=True)
489     csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
490     with open(csv_path, "w", newline="", encoding="utf-8") as f:
491         writer = csv.writer(f)
492         writer.writerow(["start", "end"])
493         for start, end in intervals:
494             writer.writerow([start, end])
495
496     manifest.append({
497         "video_id": video.video_id,
498         "sport": sport.value,
499         "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
500         "local_path": video.local_path,
501         "video_url": video.video_url,
502         "match_name": video.match_name,
503         "fps": video.fps,
504         "resolution": video.resolution,
505         "license_type": video.license_type.value,
506         "is_commercial": video.is_commercial,
507         "status": video.status.value,
508         "num_segments": len(intervals),
509     })
510
511     os.makedirs(out_dir, exist_ok=True)
512     manifest_doc = {
513         "commercial_only": commercial_only,
514         "statuses": [s.value for s in statuses],
515         "total_videos": len(manifest),
516         "total_segments": sum(m["num_segments"] for m in manifest),
517         "eval_sets": manifest,
518     }
519     manifest_path = os.path.join(out_dir, "manifest.json")
520     with open(manifest_path, "w", encoding="utf-8") as f:
521         json.dump(manifest_doc, f, indent=2)
522
523     print(f"\n=== Exported eval/training set to '{out_dir}' ===")
524     if manifest:
525         summary = [[m["sport"], m["video_id"], m["num_segments"],
526                     "YES" if m["is_commercial"] else "NO",
527                     os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
528                     for m in manifest]
529         print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
530         print(f"\nVideos exported : {len(manifest)}")
531         print(f"Total segments : {manifest_doc['total_segments']}")
532         if skipped_noncomm:
533             print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
534         if skipped_empty:
535             print(f"Skipped (no annotations) : {skipped_empty}")
536         print(f"Manifest : {manifest_path}\n")
537
538     def convert_cvat(self, args):
539         """Convert a vendor CVAT rally export into our canonical per-rally CSV.

```

**19. user (2026-06-08T08:44:13.550Z)**

C:\Users\avidu\Projects\Annotation Setup\rally\_review.csv  
C:\Users\avidu\Projects\Annotation Setup\golden\_labels\_badminton.csv  
C:\Users\avidu\Projects\Annotation Setup\Collect\Adarsh and Avi.rallies.csv

**20. assistant (2026-06-08T08:44:16.460Z)**

Let me inspect the actual partial CSV (`Adarsh and Avi.rallies.csv`) and the richer `rally\_review.csv` to ground exactly which columns map and what partial/fractional/missing-end realities appear in real data.

#### 21. user (2026-06-08T08:44:17.420Z)

```
1 rally_number,start_time,end_time,ending_reason,sport
2 1,37.434,72.194,winner,badminton
3 2,79.936,112.954,winner,badminton
4 3,130.193,134.947,winner,badminton
5 4,139.697,144.944,unforced_error,badminton
6 5,152.196,157.198,unforced_error,badminton
7 6,161.694,165.950,unforced_error,badminton
8 7,170.817,176.327,forced_error,badminton
9 8,182.829,186.083,unforced_error,badminton
10 9,193.576,199.831,forced_error,badminton
```

#### 22. user (2026-06-08T08:44:17.896Z)

```
1 rally,start_mmss,end_mmss,dur_s,shots,ending_reason,ending_consistent,notes,auto_flag,VE
RIFIED(Y/N),true_start_mmss,true_end_mmss,true_ending,reviewer_comment
2 1,0:03.003,0:21.021,18.0,15,other,yes,net,,,,,,,,
3 2,0:02.502,0:32.532,30.0,2,forced_error,yes,,OVERLAP;INFLATED?,,,,,
4 3,0:44.544,0:51.051,6.5,5,unforced_error,yes,,,,,,,,
5 4,1:02.062,1:07.067,5.0,4,unforced_error,yes,,,,,,,,
6 5,1:12.072,1:16.076,4.0,4,unforced_error,yes,,,,,,,,
7 6,1:23.083,1:29.089,6.0,4,winner,yes,,,,,,,,
8 7,1:35.596,1:51.612,16.0,2,winner,yes,,INFLATED?,,,,,
9 8,1:58.118,1:59.620,1.5,1,service_fault,yes,,,,,,,,
10 9,2:06.627,2:11.131,4.5,4,winner,yes,,,,,,,,
11 10,2:18.138,2:21.141,3.0,2,unforced_error,yes,,,,,,,,
12 11,2:29.649,2:34.154,4.5,3,winner,yes,,,,,,,,
13 12,2:43.163,2:45.165,2.0,1,service_fault,yes,,,,,,,,
14 13,2:53.673,2:58.178,4.5,2,winner,yes,,,,,,,,
15 14,3:03.183,3:06.186,3.0,3,winner,yes,,,,,,,,
16 15,3:12.692,3:16.196,3.5,3,forced_error|other,NO,net,,,,,,,,
17 16,3:25.706,3:43.223,17.5,2,other|winner,NO,net,INFLATED?,,,,,
18 17,3:56.236,3:57.738,1.5,1,winner,yes,,,,,,,,
19 18,4:03.243,4:12.252,9.0,9,winner,yes,,,,,,,,
20 19,4:22.762,4:25.265,2.5,2,winner,yes,,,,,,,,
```

#### 23. assistant (2026-06-08T08:44:26.728Z)

I now have everything needed. The lean rally-annotator CSV (`Adarsh and Avi.rallies.csv`) is already directly importable today ? its columns (`rally\_number,start\_time,end\_time,ending\_reason,sport`) match the `import-local` CSV reader exactly, with integer rally numbers, decimal seconds, and complete (start, end) pairs. The rich `rally\_review.csv` is the format that needs handling work (mm:ss times, `rally` not `rally\_number`, fractional numbers, pipe-delimited reasons, no `start\_time`/`end\_time`).

Let me verify one more thing ? the export interval ordering, since partial coverage means the eval needs a labeled-window, and check that `get\_segment\_intervals` orders by time not rally\_number.

#### 24. user (2026-06-08T08:44:28.011Z)

```
527 def get_segment_intervals(self, video_id: str, sport: SportType) -> List[tuple]:
528     """Return chronologically-ordered (start_time, end_time) tuples for a video,
529     regardless of sport. This is the sport-agnostic view used to export
530     indexer-ready evaluation/training labels.
531
532     Sports that have no annotation table yet (e.g. tabletennis, pickleball)
533     yield an empty list rather than raising.
534     """
```

```

535         mapping = SEGMENT_TABLES.get(sport)
536         if mapping is None:
537             return []
538         table, order_by = mapping # both come from the trusted map above, never user
input
539         with self._get_connection() as conn:
540             cursor = conn.cursor()
541             cursor.execute(
542                 f"SELECT start_time, end_time FROM {table} "
543                 f"WHERE video_id = ? ORDER BY {order_by}",
544                 (video_id,),
545             )
546             return [(row["start_time"], row["end_time"]) for row in cursor.fetchall()]

```

## 25. assistant (2026-06-08T08:44:30.528Z)

I have full grounding now. Let me check the `order\_by` for badminton to confirm whether truncated fractional rally numbers could corrupt export ordering.

## 26. user (2026-06-08T08:44:32.321Z)

```

16: SEGMENT_TABLES = {
17:     SportType.BADMINTON: ("badminton_rallies", "rally_number"),
20:     SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
21:     SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
77:         rally_number INTEGER NOT NULL,
85:         PRIMARY KEY (video_id, rally_number),
128:         rally_number INTEGER NOT NULL,
136:         PRIMARY KEY (video_id, rally_number),
145:         rally_number INTEGER NOT NULL,
153:         PRIMARY KEY (video_id, rally_number),
222:         video_id, rally_number, start_time, end_time,
227:         rally.rally_number,
306:         video_id, rally_number, start_time, end_time,
311:         rally.rally_number,
334:         video_id, rally_number, start_time, end_time,

```

[Showing results with pagination = limit: 15]

## 27. assistant (2026-06-08T08:44:36.680Z)

This confirms the critical bug surface: badminton export orders by `rally\_number`, which is the PK `(video\_id, rally\_number)` as INTEGER. If two fractional rallies (e.g., `7` and `7.5`) both truncate to `7`, the second silently overwrites the first via the UPSERT, AND ordering by rally\_number can diverge from chronological time. But for the lean rally-annotator CSV, rally numbers are already integers ? so the smallest path is to ship the lean format today and guard the fractional case.

I have everything I need to produce concrete, grounded recommendations.

## 28. user (2026-06-08T08:45:35.323Z)

Structured output provided successfully

## 29. assistant (2026-06-08T08:45:41.047Z)

I've completed the analysis and returned the structured recommendation. Key grounded findings:

- The lean rally-annotator CSV (`Adarsh and Avi.rallies.csv`) is **already importable today** with zero collector changes ? its columns match the `import-local` CSV reader (`curate.py:232-250`) exactly.
- The three landmines for partial human data, each with a smallest fix: fractional `rally\_number` truncation/PK-collision (`parsing.py:17` + `db.py:210-237`), missing `end\_time` aborting the whole import (`parsing.py:22-28`), and partial coverage scoring (already solved

downstream by `eval\_partial\_labels.py:47-78` via `--window-end`).

- The rich `rally\_review.csv` (mm:ss times, fractional `rally`, pipe-delimited reasons) is correctly scoped OUT ? route via a one-off converter rather than expanding `import-local`.